11-matchers-included-in-rspec-expectations/02/higher_order_matchers.rb
```
def self.evens_up_to(n = 0)
  0.upto(n).select(&:odd?)
end

expect(evens_up_to).to all be_even
```

This method generates odd numbers instead of evens, but our expectation didn't fail. We forgot to pass an argument to evens_up_to, and it returned an empty array. One solution is to use a compound matcher to ensure the array is non-empty:

11-matchers-included-in-rspec-expectations/02/higher_order_matchers.rb
```
RSpec::Matchers.define_negated_matcher :be_non_empty, :be_empty

expect(evens_up_to).to be_non_empty.and all be_even
```

We're using another RSpec feature, define_negated_matcher, to create a new be_non_empty matcher that's the opposite of be_empty. We'll learn more about define_negated_matcher in *Negating Matchers*, on page 219.

Now, the expectation correctly flags the broken method as failing:

```
expected `[].empty?` to return false, got true
```

RSpec's all matcher uses Ruby's Enumerable#all? under the hood. You may be wondering whether or not RSpec has matchers for the other similar Enumerable methods, like any or none. It doesn't, because this would lead to nonsensical code such as expect(numbers).to none be_even. Instead, you can build up easier-to-read matchers using to include or not_to include.[6]

match

If you call JSON or XML APIs, you often end up with deeply nested arrays and hashes. The match matcher is a Swiss army knife for this kind of data.

As you did with eq, you provide a data structure that's laid out like the result you're expecting. match is more flexible, however. You can substitute a matcher for any array element, or for any hash value, at any level of nesting:

11-matchers-included-in-rspec-expectations/02/higher_order_matchers.rb
```
children = [
  { name: 'Coen',   age: 6 },
  { name: 'Daphne', age: 4 },
  { name: 'Crosby', age: 2 }
]
```

6. That is the question.